Informe de Arquitectura, Flujos y Funcionamiento

Proyecto #3 (Unidad III) — Monitoreo de Material Particulado MP 2.5 μm

Interfaces Gráficas (INFO1128) — Universidad Católica de Temuco

Fabián García

Profesor: Alberto Caro — Ayudante: Daniela Romero

Noviembre 2026

---

1. Arquitectura General y Flujo de Datos de Punta a Punta

El proyecto consiste en un sistema de monitoreo en tiempo real para medir y graficar la concentración de **Material Particulado fino** ($MP_{2,5}\,\mu g/m^3$), conectando cuatro programas independientes que se comunican entre sí paso a paso:

1. **Simulador Serial en Python (`2_simulador_serial/simulador_serial.py`):** Hace el papel del sensor físico de calidad del aire. Una vez por segundo genera una medición simulada siguiendo una **Distribución Normal(80, 5)** y la transmite como texto por el puerto serial `COM2` a 9600 baudios (configuración `8N1`).
2. **Puente Serial Virtual en VSPE (`Connector` en `COM2`):** En Windows, un puerto serial normal solo puede ser usado por un programa a la vez; si un segundo programa intenta abrirlo, el sistema lanza error de acceso denegado. Al crear un dispositivo tipo `Connector` sobre `COM2` en VSPE, se habilita un puerto virtual compartido donde Python puede escribir y Lazarus puede leer simultáneamente sin bloquearse.
3. **Visor Gráfico en Lazarus (`3_visor_mp25_lazarus/main.pas`):** Es la ventana principal del proyecto (*"By Alberto Caro"*). Escucha el puerto `COM2` con el componente `TLazSerial` y, cada vez que recibe una muestra, desplaza la curva un lugar hacia la izquierda (**Scrolling Left** sobre 30 muestras fijas del `0` al `29`). Además permite activar o desactivar la grilla (`On/off`), rotar entre 6 paisajes de fondo (`Click`), guardar una foto del gráfico en `captura.png` (`Save`) y reenviar inmediatamente el valor medido por red local hacia el puerto TCP `9000`.
4. **Servidor TCP/IP en Consola Lazarus (`1_server_tcp_lazarus/server_tcp.lpr`):** Es un programa de consola puro que escucha en el puerto TCP `9000`. Cada vez que el visor le envía un valor de $MP_{2,5}$, el servidor toma la fecha y hora actual del computador, guarda el registro en la base de datos SQLite `aire.db` (dentro de la tabla `Aire`) y muestra por consola la fila recién insertada.
5. **Inspección en HeidiSQL y Visualización Web con QuickChart (`4_quickchart_web/app_quickchart.p`** Como la base de datos SQLite trabaja en modo concurrente (`WAL`), podemos abrir `aire.db` en **HeidiSQL 12.3** para ver cómo entran los registros en vivo segundo a segundo, y al mismo tiempo levantar un servidor web en **Flask** (puerto `5000`) que toma las últimas 20 mediciones de la tabla `Aire` y genera una gráfica en la nube usando la API de **QuickChart.io**.

Tramo	Medio	Configuración	Qué viaja	Función en el flujo
Python → Visor	Serial (`COM2`)	9600 Bps, 8N1, 1 seg	`<valor>\r\n`	Entrega la lectura del sensor al sor gráfico.
Visor → Server TCP	Socket TCP	`127.0.0.1:9000`	`<valor>\n`	Reenvía el dato al servidor de c sola.
Server TCP → SQLite	Archivo local	Modo `WAL` activo	`fecha, hora, mp25`	Guarda el historial en la tabla `A` de `aire.db`.
HeidiSQL → SQLite	`sqlite3.dll`	Lectura en vivo	`SELECT * FROM Aire`	Permite revisar la tabla en tier real sin bloquear al servidor.
Flask → QuickChart	HTTPS (`POST`)	Puerto 5000 → Nube	JSON de Chart.js	Pinta las últimas 20 lecturas QuickChart.io y redirige (`302`)

Tabla 1: Resumen del recorrido del dato entre los módulos del proyecto.

2. Simulación del Sensor y Comunicación Serial (`simulador_serial.py`)

Toda la lógica del sensor está dentro de la función `main()` del archivo `2_simulador_serial/simulador_serial.py`

- **Por qué se modela con Distribución Normal(80, 5) (`random.gauss(80, 5)`):** En el aire real, la cantidad de partículas en suspensión no salta de un extremo a otro en un segundo. Si usáramos un aleatorio simple como `randint(0, 200)`, la curva daría saltos irreales (por ejemplo, de 15 a 190 en un segundo). En cambio, la distribución normal centrada en media $\mu = 80$ con desviación estándar $\sigma = 5$ hace que la mayoría de los valores se agrupen cerca de $80\,\mu g/m^3$ y se muevan suavemente entre 70 y $90\,\mu g/m^3$ (y casi nunca salgan de 65 a 95), tal como pide el punto 5 de la pauta.
- **Por qué la trama lleva salto de línea (`\r\n`) y se usa `puerto.flush()`:** Al enviar por serial se transforma el número en texto terminado en `\r\n` y se codifica a bytes (`.encode(ütf-8")`). Ese salto de línea final es indispensable porque en Lazarus el componente `PuertoSerial` (`TLazSerial`) tiene activada la opción `RcvLineCRLF = True`: gracias a eso, Lazarus espera hasta ver el salto de línea antes de entregar el dato completo, evitando leer números cortados por la mitad. Inmediatamente después de escribir con `puerto.write()`, se llama a `puerto.flush()` para obligar a Windows a vaciar el búfer serial al instante, manteniendo el ritmo exacto de 1 muestra por segundo (`time.sleep(1.0)`).

3. Visor Gráfico y Dibujo Dinámico en Lazarus (`main.pas` y `main.lfm`)

El formulario principal (`TFormPrincipal` en `3_visor_mp25_lazarus/main.pas`) concentra el trabajo de interfaz gráfica con el componente `GraficoMP25` (`TChart`) y su serie `SerieMP25` (`TLineSeries`).

3.1. Efecto de Movimiento Continuo (*Scrolling Left* en 30 muestras)

En Lazarus un gráfico no avanza solo en el tiempo; para lograr el efecto de desplazamiento hacia la izquierda se combinan dos procedimientos:

- **Recepción y corrimiento de datos (`PuertoSerialRxData`):** El programa guarda en memoria un arreglo fijo de 30 casilleros llamado `historial_mp25[0..29]` (que parte inicializado en cero en `FormCreate`). Cada vez que entra una línea por el puerto `COM2`, se valida que sea un número entero con `TryStrToInt` y un ciclo `for` mueve todos los elementos un puesto hacia la izquierda

(`historial_mp25[j] := historial_mp25[j + 1]`). Con esto, el dato más viejo (posición `0`) sale de la ventana y el dato recién llegado se guarda en la última posición de la derecha (posición `29`).

- **Borrado y repintado rápido (`ActualizarGrafico`):** Apenas se actualiza el arreglo, `ActualizarGrafico` congela momentáneamente el refresco de la serie (`BeginUpdate`), borra los puntos anteriores (`SerieMP25.Clear`), vuelve a insertar los 30 puntos del `0` al `29` con `SerieMP25.AddXY(j, historial_mp25[j])` y libera el dibujo (`EndUpdate`). Como la ventana tiene activado `DoubleBuffered = True`, ese borrado y repintado ocurre en memoria antes de mostrarse en el monitor, por lo que el ojo ve que la curva camina suavemente hacia la izquierda cada segundo.

3.2. Por qué se Fijan los Ejes y Cómo Funcionan las Propiedades de `TChart`

Al arrancar el programa en `FormCreate`, se configuran por código tanto los límites del gráfico como las propiedades visuales exigidas en los puntos 9, 10 y 11 del PDF:

- **Piso y techo fijos (`Extent`):** Por defecto, `TChart` auto-ajusta sus ejes según los valores que tenga en ese momento. Si no lo bloqueáramos, cada vez que entrara un valor distinto la escala vertical saltaría y toda la curva temblaría. Por eso en `FormCreate` fijamos el eje Y entre `0` y `200` ($\mu g/m^3$) y el eje X entre `0` y `29` muestras, dejando la estructura del visor firme como un instrumento de medición real.
- **Trazo y vértices de la curva (`SeriesColor`, `LinePen` y `Pointer`):** `SeriesColor` y `LinePen` definen el color rojo (`clRed`) y el grosor de 2 píxeles de la línea que une las muestras, mientras que `Pointer` dibuja el pequeño cuadrado azul/celeste (`psRectangle`) sobre cada vértice para distinguir dónde cae exactamente cada segundo.
- **Cajitas de valores sobre cada punto (`Marks`, `Frame` y `LabelBrush`):** La propiedad `Marks` con estilo `smsValue` hace que cada punto muestre arriba su valor numérico. El detalle visual más importante aquí es el por qué de `LabelBrush` y `Frame`: como el fondo del gráfico es una ilustración a todo color (con árboles, montañas y cielo), si escribiéramos el número suelto no se leería nada. Al darle un relleno sólido claro (`LabelBrush`) rodeado por un borde negro (`Frame`), cada número queda dentro de una cajita opaca que se lee perfecto sobre cualquier imagen de fondo.
- **Títulos y Leyenda (`Title`, `BottomAxis.Title`, `LeftAxis.Title` y `Legend`):** Configuran el título superior (*"Material Particulado 2.5 um"*), el subtítulo inferior (*"Sampling MP 2.5um"*), la unidad vertical rotada 90 grados (*"[ug/m3]"*) y el recuadro de leyenda arriba a la derecha (*"MP 2.5 um"*).

3.3. Cómo se Pintan los 6 Paisajes de Fondo sin Parpadeo (`DibujarFondoGrafico`)

El componente `TChart` dibuja su contenido por capas: primero pinta el recuadro de fondo (*BackWall*), encima dibuja la grilla punteada y los ejes, y al final traza la curva roja con sus cajitas de números.

Para colocar los 6 paisajes (`img1.png` a `img6.png`) detrás de la curva sin tapar la grilla ni los datos, enganchamos nuestro procedimiento `DibujarFondoGrafico` al evento `OnAfterDrawBackWall` del gráfico:

- **El problema de rendimiento:** Si cada vez que entra un dato serial (cada 1 segundo) el programa tuviera que tomar la imagen PNG original y estirarla (`StretchDraw`) al ancho y alto del gráfico, gastaría procesador innecesariamente y la imagen parpadearía.
- **La solución con caché en memoria RAM (`bitmap_fondo_cache`):** Al iniciar en `FormCreate` cargamos las 6 fotos en el arreglo `imagenes_fondo[1..6]` (partiendo en el paisaje 6, igual que en el video del profesor). Cuando se dibuja el gráfico por primera vez, estiramos el paisaje actual una sola vez dentro de un objeto en memoria RAM llamado `bitmap_fondo_cache`. En todos

los segundos siguientes, `DibujarFondoGrafico` ve que el tamaño no ha cambiado y simplemente estampa esa imagen ya lista desde la RAM (`ACanvas.Draw`), lo que toma menos de un milisegundo y evita cualquier parpadeo.

3.4. Botones de la Interfaz (`Save`, `On/off`, `Click`), Íconos y Propiedad `Hint`

Los tres botones exigidos en la pauta son componentes `TBitBtn` cuyos íconos (`Glyph`) se cargan dinámicamente en `FormCreate` usando rutas relativas (`ObtenerCarpetaBase`), junto con el ícono de la ventana (`sms2.bmp` en `Self.Icon`). Todos tienen activada su propiedad `Hint` y `ShowHint = True` para mostrar el cartel explicativo al pasar el cursor por encima:

- **Botón `Save` (`BtnSaveClick`, ícono `camara.png`):** Usa el método nativo `GraficoMP25.SaveToFile(TPortable ...)` para exportar todo lo que se ve actualmente en el gráfico (paisaje, grilla, títulos y curva) directamente al archivo `captura.png` (y además guarda una copia con fecha y hora en la carpeta `capturas/`).
- **Botón `On/off` (`BtnOnOffClick`, ícono `f02.bmp`):** Conmuta la variable booleana `grilla_activa` (`grilla_activa := not grilla_activa`) y se la asigna a `LeftAxis.Grid.Visible` y `BottomAxis.Grid.Visib` encendiendo o apagando la cuadrícula punteada al instante.
- **Botón `Click` (`BtnClickFondoClick`, ícono `pale.png`):** Avanza el contador `indice_fondo` del `1` al `6` (volviendo a `1` después del `6`), limpia las dimensiones de `bitmap_fondo_cache` para obligar a que estire la nueva foto y llama a `GraficoMP25.Invalidate` para refrescar el paisaje en pantalla.
- **Controles de conexión (`ChkConectarSerial`, `ChkEnviarTCP` y `TimerReconexion`):** Para comodidad durante las pruebas y la defensa, las casillas de puerto serial y envío TCP parten activadas desde el arranque. Si al abrir el visor aún no se le ha dado *Play* al `COM2` en VSPE, `TimerReconexionTimer` reintenta conectar en silencio cada 2 segundos hasta que el puerto esté disponible.

3.5. Envío por Socket TCP sin Congelar la Ventana (`EnviarMP25PorTCP`)

Cada vez que `PuertoSerialRxData` recibe un valor de $MP_{2,5}$ y actualiza el gráfico, llama a la función `EnviarMP25PorTCP(valor_mp25)` para mandarle el dato al servidor TCP en `127.0.0.1:9000`.

Por qué se usa un socket no bloqueante de 80 ms: En Windows, si un programa gráfico intenta hacer un `connect` normal hacia un puerto cuyo servidor está apagado, Windows deja esperando al programa cerca de 2 segundos antes de dar error; eso haría que toda la ventana de Lazarus se quedara pegada (*"No responde"*). Para evitarlo, en `EnviarMP25PorTCP` ponemos el socket en modo no bloqueante (`ioctlsocket` con `FIONBIO`) y usamos `select` con un tiempo de espera de apenas 80 milisegundos. Si el servidor TCP está encendido, conecta y envía el número (`send`) en menos de 1 milisegundo; y si el servidor TCP está apagado, a los 80 milisegundos avisa en la etiqueta de estado (*"TCP: Sin Servidor"*) y la curva sigue moviéndose con total fluidez.

4. Servidor TCP en Consola y Base de Datos SQLite (`server_tcp.lpr`)

El servidor (`1_server_tcp_lazarus/server_tcp.lpr`) cumple el punto 12 del PDF: recibir cada medición por TCP/IP en modo consola, mostrarla en pantalla y guardarla con fecha y hora en la tabla `Aire` de SQLite. Se divide en tres partes principales:

- **Creación de `aire.db` y por qué es clave el modo `WAL` (`IniciarBaseDatos`):** Al arrancar el servidor, `IniciarBaseDatos` abre o crea el archivo `aire.db` y ejecuta `CREATE TABLE IF NOT`

`EXISTS Aire (id, fecha, hora, mp25)`. Justo antes de crear la tabla, ejecuta la instrucción `PRAGMA journal_mode=WAL;` (*Write-Ahead Logging*).

El por qué de `WAL`: En el modo tradicional de SQLite, cuando un programa escribe en la base de datos bloquea el archivo entero. Como nuestro servidor inserta un dato nuevo cada 1 segundo, si abriéramos `aire.db` al mismo tiempo en **HeidiSQL** o desde el servidor web **Flask**, chocarían y saldría el error `database is locked`. Con el modo `WAL` activado, las inserciones nuevas se van guardando en un archivo auxiliar (`aire.db-wal`) sin bloquear las lecturas, permitiendo que el servidor TCP escriba cada segundo y que nosotros consultemos y refresquemos HeidiSQL o Flask en vivo sin que jamás se bloquee.

- **Escucha de red en el puerto 9000 (`EjecutarServidorTCP`):** Usa la librería nativa de sockets de Windows (`WinSock2`). Crea el `socket_servidor`, activa la opción `SO_REUSEADDR` (para que si cerramos y volvemos a abrir el servidor no diga que el puerto 9000 sigue ocupado), lo enlaza al puerto `9000` con `bind`, lo pone en escucha con `listen` y entra en un ciclo infinito `while True` esperando conexiones del visor con `accept`. Cuando entra el paquete del visor, lee los bytes con `recv` y le pasa la línea a `ProcesarLinea`.
- **Estampado de Fecha/Hora e inserción en la tabla `Aire` (`ProcesarLinea` y `GuardarEnSQLite`):** `ProcesarLinea` extrae el número entero con `ExtraerNumero`, obtiene la fecha (`yyyy-mm-dd`) y la hora (`hh:nn:ss`) del reloj del sistema usando `FormatDateTime` y llama a `GuardarEnSQLite`. Adentro, `GuardarEnSQLite` usa un componente `TSQLQuery` con parámetros (`:f`, `:h`, `:m`) para hacer el `INSERT INTO Aire`, confirma la transacción con `Commit` y devuelve el `id` generado para imprimir en la consola el mensaje completo con número de registro, fecha, hora y valor de $MP_{2,5}$. En la misma carpeta está el archivo `sqlite3.dll` (de 64 bits), necesario para que tanto el ejecutable de Lazarus como HeidiSQL puedan abrir motores SQLite3 en Windows de 64 bits.

5. Auditoría en Tiempo Real (HeidiSQL) y Gráfica Web (`app_quickchart.py`)

Para cerrar el flujo de datos y demostrar que la base de datos realmente se está alimentando en vivo, el proyecto cuenta con dos vías de visualización externa:

- **Verificación en tiempo real con HeidiSQL 12.3 Portable (Punto 13 del PDF):** En HeidiSQL se crea una sesión seleccionando el tipo de red **SQLite**, indicando como librería el archivo `1_server_tcp_lazarus/sqlite3.dll` y como archivo de base de datos `1_server_tcp_lazarus/aire.db`. Una vez abierta la base, podemos ir a la pestaña **Datos** de la tabla `Aire` y presionar **F5** (o ejecutar en la pestaña **Consulta** un `SELECT * FROM Aire ORDER BY id DESC LIMIT 20;`) para ver cómo se agregan las filas segundo a segundo con la misma fecha, hora y valor que muestra la consola del servidor TCP.
- **Servidor Web Flask y Gráfica en la Nube con QuickChart.io (`4_quickchart_web/app_quickchart.py`):** Siguiendo lo explicado por el profesor en la clase complementaria del 3 de noviembre, el script `app_quickchart.py` levanta un servidor Flask en el puerto `5000` que se conecta de forma relativa a `../1_server_tcp_lazarus/aire.db` mediante `obtener_conexion_bd()` y ofrece tres rutas:
  1. **Ruta principal `/` (función `index()`):** Muestra un panel web con el estado de conexión a `aire.db`, el total de muestras acumuladas en la tabla `Aire`, el último valor registrado y una tabla con las últimas 15 lecturas que se auto-refresca cada 3 segundos.
  2. **Ruta `/grafica` (función `obtener_grafica()`):** Consulta las últimas 20 filas de la tabla `Aire` (`ORDER BY id DESC LIMIT 20`) y luego invierte la lista con `filas.reverse()`. Se invierte porque la consulta SQL trae desde la más nueva hasta la más antigua, pero en un gráfico de tiempo necesitamos que el eje X avance de izquierda (más antiguo) a derecha (más reciente). Con esas 20 horas y valores arma el diccionario `configuracion_grafica` con la estructura de **Chart.js** (curva roja con puntos azules), lo envía mediante una petición `requests.post`

a `https://quickchart.io/chart/create`, recibe la URL de la imagen generada y redirige automáticamente al navegador con `redirect(url_grafica, code=302)`.

3. **Ruta `/datos` (función `obtener_datos()`):** Devuelve en formato JSON las últimas 30 filas de la tabla `Aire` para poder inspeccionarlas directamente desde el navegador.